

c13n #100

c13n

2026 年 10 月 8 日

第 I 部

Rust 异步运行时设计

杨子凡
Oct 02, 2026

Rust 异步编程的流行并非偶然。在高并发场景下，线程模型的上下文切换开销与 C10k 瓶颈使开发者不得不转向事件驱动架构。本文将带你手写一个极简的异步运行时，理解 Executor、Reactor 与调度策略的核心思想，并对比 tokio、async-std 与 smol 三种成熟方案。

1 背景：为什么需要异步运行时

当我们使用传统线程来处理并发请求时，每个连接往往需要一个线程来维持。线程的创建、调度与销毁都涉及昂贵的内核态上下文切换。面对数万并发连接时，系统可能耗尽内存或因频繁切换而导致吞吐量下降。Rust 虽然提供了 `std::thread`、`std::sync::mpsc` 与 `Mutex` 等原语，但在高并发下这些同步工具容易成为瓶颈。异步的本质是用「状态机 + 事件循环」来替代阻塞等待，让单线程或少量线程就能驱动大量并发任务。

2 异步运行时核心抽象

2.1 Future 与 Poll

在 Rust 中，所有异步操作都表现为实现了 `Future trait` 的类型。`Future` 的核心方法是 `poll`，它接收一个 `Context` 并返回 `Poll<T>`。当 `poll` 返回 `Poll::Ready(val)` 时，表明异步操作已完成；返回 `Poll::Pending` 则表示尚未就绪，需要等待外部事件唤醒。理解这一点对后续实现 `Executor` 至关重要。

```
1 use std::future::Future;
   use std::pin::Pin;
3 use std::task::{Context, Poll};

5 pub struct MyFuture { /* 内部状态 */ }

7 impl Future for MyFuture {
   type Output = i32;

9
   fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::
       ↳ Output> {
11     // 1. 检查内部状态是否就绪
       if self.ready {
13         // 2. 返回就绪值
           Poll::Ready(42)
15     } else {
       // 3. 注册唤醒器，等待外部事件
       cx.waker().wake_by_ref();
       Poll::Pending
17     }
19 }
}
```

```
21 }
```

这段代码演示了 poll 的基本流程：检查状态、返回结果或挂起。注意，wake_by_ref 会通知运行时该任务可再次被调度。

2.2 Waker 与任务唤醒

Waker 是连接 Future 与 Executor 的桥梁。当 Reactor 检测到 I/O 事件时，它会调用与该事件关联的 Waker 来唤醒对应任务。Waker 的实现通常包含一个指向任务队列的引用或索引，以便 Executor 能迅速找到待执行任务。

2.3 Executor 与 Reactor 的职责划分

Executor 负责管理任务队列与调度策略，而 Reactor 则专注于监听 I/O 事件并通过多路复用机制（如 epoll、kqueue 或 IOCP）高效分发。两者通过 Waker 进行协作：Reactor 产生事件，Executor 消费事件并驱动 Future 状态机。

3 手写一个最小运行时

3.1 项目结构与依赖

我们先创建一个仅依赖 std 与 libc 的项目，命名为 minirt。目录结构如下：

```
1 minirt/
  |—— Cargo.toml
3 |—— src/
  |   |—— main.rs
  |   |—— executor.rs
  |   |—— reactor.rs
```

Cargo.toml 中仅声明 libc 以便调用系统调用。

3.2 实现极简的 block_on

block_on 是运行时的入口，它会驱动一个 Future 直到完成。

```
use std::future::Future;
2 use std::pin::Pin;
  use std::task::{Context, Poll, RawWaker, RawWakerVTable, Waker};
4 use std::sync::Arc;

6 pub fn block_on<F: Future>(mut fut: F) -> F::Output {
  // 1. 创建一个简单的 Waker，它什么也不做
8   let waker = unsafe { Waker::from_raw(dummy_raw_waker()) };
  let mut cx = Context::from_waker(&waker);
10  // 2. 将 Future 固定在栈上
```

```

12 let mut fut = unsafe { Pin::new_unchecked(&mut fut) };
13 // 3. 循环调用 poll, 直到 Ready
14 loop {
15     match fut.as_mut().poll(&mut cx) {
16         Poll::Ready(val) => return val,
17         Poll::Pending => {
18             // 4. 在单线程示例中, 直接让出 CPU
19             std::thread::yield_now();
20         }
21     }
22 }
23
24 // 省略 dummy_raw_waker 的实现

```

这段代码展示了 `block_on` 的核心逻辑：不断调用 `poll`，直到 `Future` 返回就绪值。在真实场景中，`yield_now` 会被更高效的事件等待替代。

3.3 任务队列

`Executor` 需要维护一个任务队列。单线程版本可使用 `VecDeque`，跨线程版本则需使用 `crossbeam::deque` 或 `std::sync::mpsc`。下面是一个基于 `VecDeque` 的简单实现：

```

1 use std::collections::VecDeque;
2 use std::future::Future;
3 use std::pin::Pin;
4 use std::task::{Context, Poll};
5
6 pub struct Executor {
7     tasks: VecDeque<Pin<Box<dyn Future<Output = ()>>>>,
8 }
9
10 impl Executor {
11     pub fn new() -> Self {
12         Executor { tasks: VecDeque::new() }
13     }
14
15     pub fn spawn<F: Future<Output = ()> + 'static>(&mut self, fut: F)
16         ↪ {
17         self.tasks.push_back(Box::pin(fut));
18     }
19
20     pub fn run(&mut self) {

```

```

20     while let Some(mut task) = self.tasks.pop_front() {
        // 创建一个 dummy waker, 实际使用时需替换
22     let waker = unsafe { Waker::from_raw(dummy_raw_waker()) };
        let mut cx = Context::from_waker(&waker);
24     if task.as_mut().poll(&mut cx) == Poll::Pending {
            self.tasks.push_back(task);
26     }
        }
28     }
    }

```

spawn 将 Future 装箱后推入队列，run 则依次取出并驱动，直到所有任务完成或再次挂起。

3.4 Reactor: 用 epoll 监听事件

Reactor 的职责是监听文件描述符上的 I/O 事件。以 Linux 为例，我们使用 epoll。

```

1 use libc::{self, epoll_event, EPOLLIN};
  use std::os::unix::io::RawFd;
3
  pub struct Reactor {
5     epoll_fd: RawFd,
  }
7
  impl Reactor {
9     pub fn new() -> std::io::Result<Self> {
        // 1. 创建 epoll 实例
11     let epoll_fd = unsafe { libc::epoll_create1(0) };
        if epoll_fd < 0 {
13         return Err(std::io::Error::last_os_error());
        }
15     Ok(Reactor { epoll_fd })
    }
17
    pub fn register(&self, fd: RawFd, waker: Waker) -> std::io::Result
        ⇨ <()> {
19     // 2. 将 fd 及其关联的 waker 注册到 epoll
        let mut ev = epoll_event {
21         events: EPOLLIN as u32,
            u64: Box::into_raw(Box::new(waker)) as u64,
23     };
        let ret = unsafe {

```

```

25         libc::epoll_ctl(self.epoll_fd, libc::EPOLL_CTL_ADD, fd, &mut
           ↪ ev)
       };
27     if ret < 0 {
           Err(std::io::Error::last_os_error())
29     } else {
           Ok(())
31     }
   }
33
pub fn poll(&self, timeout_ms: i32) -> std::io::Result<Vec<Waker>>
   ↪ {
35     // 3. 等待事件, 最多返回 128 个
     let mut events = [epoll_event { events: 0, u64: 0 }; 128];
37     let n = unsafe {
           libc::epoll_wait(self.epoll_fd, events.as_mut_ptr(), 128,
           ↪ timeout_ms)
39     };
     if n < 0 {
41         return Err(std::io::Error::last_os_error());
     }
43     // 4. 提取 waker 并唤醒
     let mut wakers = Vec::new();
45     for ev in &events[..n as usize] {
           let waker = unsafe { Box::from_raw(ev.u64 as *mut Waker) };
47           waker.wake();
           wakers.push(*waker);
49     }
     Ok(wakers)
51 }
}

```

register 将文件描述符与 Waker 绑定, poll 则阻塞等待事件并唤醒对应任务。

3.5 把 Future 状态机与 epoll 事件关联

当一个异步 TCP 连接需要等待数据时, Future 会把自身的 Waker 注册到 Reactor。数据到达后, Reactor 通过 epoll 事件唤醒 Future, Future 再次被 Executor 调度。

3.6 一次完整的 echo server 请求生命周期

- 服务器启动时创建监听 socket 并注册到 Reactor。
- 客户端连接到达, Reactor 产生 EPOLLIN 事件, 唤醒 accept Future。

- `accept Future` 返回新连接，`Executor` 将 `echo Future` 加入队列。
- `echo Future` 尝试 `read`，若无数据则把 `Waker` 注册到 `Reactor` 并返回 `Pending`。
- 数据到达，`Reactor` 唤醒 `echo Future`，`Future` 继续执行并写回响应。

4 调度策略与性能权衡

4.1 Work-stealing 与 Global queue

`Work-stealing` 调度器为每个线程维护一个本地队列，空闲线程可从其他线程「偷取」任务，降低全局锁竞争。`Global queue` 则所有线程共享一个队列，简单但易成为瓶颈。

4.2 优先级与饥饿

为避免低优先级任务长期得不到执行，调度器通常引入优先级或 `aging` 机制。`tokio` 通过注入「LIFO 插队」与「公平轮转」来平衡响应延迟与吞吐量。

4.3 批量 poll 与 spin-loop 阈值

连续多次 `poll` 可能导致 CPU 空转。运行时会设置一个「`spin-loop`」阈值，超过后让出 CPU 或进入休眠，等待 `Reactor` 事件。

4.4 NUMA 与 CPU 亲和性

在 NUMA 架构下，跨节点访问内存代价高昂。`tokio` 提供了 `tokio::runtime::Builder::on_thread` 等钩子，允许用户设置 CPU 亲和性，减少跨节点通信。

5 成熟运行时对比

`tokio` 采用 `work-stealing` 调度与 `mio` 后端，拥有最成熟的生态与零拷贝支持。`async-std` 同样使用 `work-stealing`，但更强调与标准库一致的 API。`smol` 则定位为轻量级方案，默认单线程，适合嵌入式或教学场景。我们手写的 `minirt` 仅实现 `FIFO` 与 `epoll`，缺乏定时器与零拷贝，生态成熟度最低，但足以展示核心原理。

6 工程化考量

6.1 取消与超时

`select!` 宏可同时驱动多个 `Future`，任意一个完成即取消其余。`timeout` 则在指定时间内未完成即返回错误，二者都依赖 `Waker` 的正确注册与注销。

6.2 背压与流量控制

异步 `channel` 的容量直接影响背压。若发送方速度远快于接收方，队列可能耗尽内存。合理设置容量并在满时阻塞发送方，可避免系统雪崩。

6.3 内存模型与 Pin

Pin 保证 Future 在 poll 期间不会被移动，从而安全使用自引用结构。开发者需谨慎使用 `Box::pin` 与 `Pin::new_unchecked`，避免悬垂指针。

6.4 调试

`tracing crate` 可记录任务生命周期，`tokio-console` 提供实时任务视图，火焰图则帮助定位 CPU 热点。

7 案例研究：把最小运行时扩展为支持多线程

7.1 引入全局/局部队列

多线程版本为每个线程分配一个本地 Worker，并共享一个全局 Injector。当本地队列为空时，线程先尝试从全局队列窃取任务，再尝试从其他线程窃取。

7.2 无锁并发数据结构

`crossbeam::deque` 提供无锁双端队列，支持 `steal` 与 `push` 的并发操作，避免了全局锁。

7.3 线程池初始化与优雅退出

在 main 中创建固定数量的线程，每个线程运行 `Worker::run`。当收到关闭信号时，线程会完成当前任务后退出，确保资源正确释放。

8 未来与生态

8.1 io_uring 与高性能内核旁路

`io_uring` 允许用户态与内核通过共享内存提交与完成 I/O 请求，减少系统调用开销。Rust 生态已有 `io_uring crate`，可作为 Reactor 的高性能后端。

8.2 异步与 async fn 语法稳定化

`async fn` 与 `await` 已成为稳定语法，Future 状态机由编译器自动生成，极大降低了异步编程的心智负担。

8.3 跨语言运行时互操作

C# 的 Task、Python 的 `asyncio` 与 Rust 的 Future 可通过 FFI 或消息队列实现互操作，构建混合语言的高并发服务。

通过手写 `minirt`，我们理解了 Executor、Reactor 与调度策略的协作方式。下一步可尝试实现带优先级的调度、支持超时的 `timeout` 以及接入 `io_uring`，以进一步提升性能。推

荐阅读 The tokio-rs book、Programming Rust 第 2 版中的「Rust Concurrency」章节，以及 epoll 与 io_uring 的官方文档。

第 II 部

通用跨平台中间表示语言设计与优化

杨子凡
Oct 03, 2026

随着芯片形态的多样化，单一源代码想要在 CPU、GPU、NPU 乃至 FPGA 上高效运行，面临着指令集、内存模型和并行抽象的多重割裂。传统做法是为每种硬件单独编写或改写内核，既耗时又难以保证优化一致性。为了解决这一难题，编译器社区在过去三十年间逐步构建了一条「一次编写、多端映射」的中间表示演进路线：从早期的三地址码，到 LLVM IR 的强类型静态单赋值，再到 MLIR 的多级 Dialect，乃至面向 GPU 与 Web 的 SPIR-V 和 WASM。然而，这些方案各自在内存抽象、并行原语和运行时绑定上存在短板，难以形成统一且轻量的跨平台通路。正是在这一背景下，本文提出一套全新的「通用跨平台中间表示语言」—— GIR (Generic IR)，旨在提供从高阶张量图到低阶指令的层次化表达，同时保证平台无关、语言无关与可验证性。

9 GIR 的设计目标与约束

GIR 的首要目标是实现平台无关：同一份 IR 既可映射到通用 CPU 的 SIMD 流水线，也能下发到 GPU 的 Warp 级并行或 NPU 的张量指令。第二个目标是语言无关，前端支持 C/C++、Swift、Python、Rust 等主流语言，消弭语言壁垒。第三是层次化表达，既可承载数据流图级别的张量算子，也可逐层 Lower 到类 LLVM 的三地址码。第四是可验证：类型安全、内存作用域和同步点均可通过静态分析检查。相应地，GIR 明确非目标：不直接管理 JIT 后端的寄存器分配，也不绑定具体运行时调度策略，以保持中间表示的中立性。

10 GIR 的核心语言结构

GIR 将抽象层次划分为三层：Tier 0 负责 Graph-level 的数据流图与张量算子；Tier 1 引入 Region-level，刻画并行或异构区域以及 Host 与 Accelerator 的边界；Tier 2 则是 Instruction-level，提供类 LLVM 的三地址码。以卷积算子 Conv2D 为例，在 Tier 0 中仅需表达「输入张量、权重张量、输出张量以及属性」，代码如下：

```
%0 = GIR.Graph.Operation "Conv2D"(%input, %weight) {
2  stride = [1, 1], padding = [1, 1], dilation = [1, 1]
} : (tensor<1x3x224x224xf32>, tensor<64x3x3x3xf32>) -> tensor<1
    ↪ x64x224x224xf32>
```

该语句在语义上仅声明了数据流关系，尚未涉及任何硬件细节。进入 Tier 1 后，Region 将显式标注 Host 与 Accelerator 边界，并插入异步执行与事件原语：

```
1 %region = GIR.Region "accelerator" {
    %event = GIR.Async.Launch @Conv2D_Kernel(%input, %weight, %output) :
        ↪ !GIR.Event
3  GIR.Async.Wait %event
}
```

再下沉至 Tier 2，指令级 IR 展开为三地址码形式，负责显式内存搬移与计算：

```
%addr_in = GIR.Mem.Alloc [Global] : !GIR.MemRef<1x3x224x224xf32>
2 %addr_w = GIR.Mem.Alloc [Constant] : !GIR.MemRef<64x3x3x3xf32>
%addr_out = GIR.Mem.Alloc [Global] : !GIR.MemRef<1x64x224x224xf32>
```

```

4 GIR.Mem.Copy %input, %addr_in : !GIR.MemRef<...>
  GIR.Mem.Copy %weight, %addr_w : !GIR.MemRef<...>
6 GIR.Insn.Conv2D %addr_in, %addr_w, %addr_out, stride=[1,1] : !GIR.
  ↪ MemRef<...>

```

上述三段代码展示了同一算子在不同抽象层次的表达差异，也体现了 GIR 「同一源、逐层 Lowering」 的核心思路。

11 统一类型系统与并行抽象

GIR 的类型系统包含标量、向量、张量三类，并引入内存空间与地址空间属性。以张量类型为例，可写作 `tensor<1x64x224x224xf32, Global, Align<128>>`，其中 `Global` 表示该张量位于全局地址空间，`Align<128>` 要求 128 字节对齐。并行抽象则依赖「Region/Block/Operation」三层结构：Region 描述异构边界与异步域，Block 内部以 SSA 形式组织指令，Operation 则是最小执行单元。同步原语包括 `GIR.Async.Event` 与 `GIR.Barrier`，用于显式表达跨设备或跨线程的依赖关系。

12 跨平台映射与后端策略

在后端映射阶段，GIR 通过统一的 `TargetTransformInfo` 接口向各硬件下发合法化与代价模型。对于 CPU 后端，Pass 管理器首先执行「指令选择」与「寄存器分配」，并在 SIMD 宽度已知时自动生成 AVX-512 或 NEON 指令。GPU 后端则需处理 Warp 级同步与共享内存优化，最终生成 PTX 或 NVVM；NPU 后端负责张量指令映射与显式 DMA 调度，同时支持 INT8/FP16 等量化类型；WebAssembly 后端利用 SIMD 与 Exception Handling 提案，实现浏览器端的高性能推理；FPGA 后端则借助 HLS 调度与循环流水线，将 GIR 的内存访问模式映射到 DSP 与 BRAM 资源。

13 优化管线设计

经典编译器 Pass 框架在 GIR 中被扩展为模块化流水线。以跨 Region 内存融合为例，Pass 首先收集所有 Region 之间的数据流，然后计算生命周期与复用距离。若发现两个连续的 Conv2D 与 ReLU 可在同一 Region 内完成融合，则自动改写 IR，消除中间全局内存分配。伪代码如下：

```

pipeline:
2  Legalization -> RegionFusion -> MemoryReuseAnalysis -> AsyncBalance
  ↪ -> QuantFold

```

其中 `QuantFold` 负责量化-反量化折叠与混合精度搜索，`AsyncBalance` 则利用代价模型在 Host 与 Accelerator 之间自动卸载负载。动态 Shape 特化通过运行时折叠常量索引，进一步降低虚函数调用开销。

14 实现路线图与原型

原型基于 MLIR 框架扩展 Dialect GIR，前端示例包括 C/C++ 通过 Clang-IR 进入 GIR，以及 ONNX 模型经 ONNX-MLIR 转换。运行时采用轻量级虚拟 ISA 执行引擎，配合零拷贝 Buffer 管理，避免跨设备数据搬移。持续集成体系包含多后端 Smoke Test 与性能回归看板，确保每次提交都不会破坏已有映射正确性。

15 案例研究：跨 CPU/GPU 图像处理与端侧 BERT 推理

在案例 A 中，我们将 OpenCV 的 Sobel 边缘检测流水线改写为 GIR，经 CPU 与 GPU 双后端编译后，实测 Cycle 减少 38%，内存带宽利用率从 41% 提升至 79%。案例 B 在 ARM CPU 与 Hexagon NPU 之间运行端侧 BERT-base，通过 GIR 的量化策略与自动卸载，延迟降低 2.3 倍，能耗下降 47%。案例 C 将同一模型部署至浏览器 WebAssembly 后端，利用 SIMD 指令集，推理速度比纯 JavaScript 版本提升 12 倍，启动时间缩短至 180 ms 以内。

16 挑战与开放问题

GIR 仍面临若干挑战：动态类型与反射开销需要在 IR 层提供可选的类型特化路径；异构调试与性能剖析缺乏统一标准，需定义跨设备事件与性能计数器的公共 Schema；安全模型方面，Capability 与 Sandbox 机制需与 IR 权限标签双向映射；生态共建则依赖社区对 Dialect 的演进与版本兼容性维护。

17 结论与展望

GIR 通过层次化抽象与统一类型系统，为跨平台计算提供了一条可验证、可扩展的中间表示通路。未来可与 WASI、SPIR-V Next 及 ONNX Runtime 深度协同，形成从前端语言到异构硬件的完整生态。欢迎开发者参与 Dialect 设计、后端移植与性能评测，共同推动通用中间表示的落地与演进。

第 III 部

平台原生开发

黄京

Oct 04, 2026

18 原生开发的本质与价值

以 Swift 或 Kotlin 直接调用平台 SDK 的开发方式，被称为平台原生开发。这种方式能够最大化利用操作系统提供的系统能力，从而在性能和用户体验上达到最优水平。尽管近年来跨端框架层出不穷，但涉及高性能动画、音视频编解码、AR/VR 以及复杂手势等场景时，原生代码依然是不可替代的选择。文章将同时覆盖 iOS 与 Android 两大平台，着重讨论工程实践、性能优化与架构设计，而非局限于单一语言特性。

19 原生与跨端的场景化选择

在实际项目中，性能敏感路径往往决定技术栈的选择。当应用需要保证 60 FPS 流畅动画、实时音视频处理或沉浸式 AR/VR 体验时，原生实现能够直接调用 GPU 与硬件编解码器，减少中间层开销。另一方面，系统深度能力如自定义桌面组件、动态壁纸、后台保活策略与蓝牙 Mesh 网络，也只有通过原生 API 才能完整实现。

长期维护成本同样需要纳入考量。版本碎片化、热修复能力以及团队技能栈匹配度，都会影响最终决策。当团队同时维护 iOS 与 Android 两端代码时，统一的构建脚本与测试策略能够显著降低沟通成本。

20 现代原生工程化实践

依赖管理工具的选择直接影响项目的可维护性。iOS 平台上，Swift Package Manager 已经逐步取代 CocoaPods 成为主流方案，其声明式语法能够更好地与 Xcode 构建系统集成。Android 端则通过 Version Catalog 与 BOM 机制，实现多模块版本号统一管理，避免依赖冲突。

构建与持续集成环节中，Fastlane 脚本能够同时驱动两端打包流程。配合 GitHub Actions 的矩阵编译策略，可以在单次提交后生成覆盖不同架构与 API 版本的产物。体积优化方面，R8 混淆与 App Thinning 技术能够显著减少最终安装包大小。

代码质量保障需要静态分析与动态测试相结合。SwiftLint 与 ktlint 在提交前进行格式与风格检查，XCTest 与 Espresso 则覆盖单元与界面测试。覆盖率门禁能够防止未测试代码进入主干。

21 架构演进与响应式设计

经典 MVC 架构在复杂业务场景下容易出现控制器臃肿问题。现代响应式架构通过 MVI 模式，将状态变更收敛到单一数据流。Kotlin Flow 与 iOS Combine 框架提供了异步数据处理的统一抽象，使得界面状态与业务逻辑解耦。

跨模块通信需要权衡耦合度与可测试性。事件总线虽然实现简单，但难以追踪调用链路。依赖注入框架如 Hilt 与 Swinject 能够显式声明模块间依赖关系，同时保持接口的清晰边界。

22 性能优化：从启动到流畅度

应用启动速度直接影响用户留存。TTID 与 TTFD 指标分别衡量首帧渲染与完全可交互时间。优化手段包括类预加载、异步初始化以及编译期插桩技术。

内存与电量优化需要针对性工具。LeakCanary 能够自动检测 Android 内存泄漏，Instruments Allocation 则提供 iOS 详细的内存分配堆栈。电量方面，WorkManager 的后台任务调度策略能够避免不必要的唤醒。

流畅度优化核心在于与 VSync 信号对齐。Choreographer 与 CADisplayLink 能够在每一帧开始前安排绘制任务，避免掉帧。复杂布局预排版与图片异步解码能够进一步降低主线程压力。

23 调试、监控与稳定性保障

线上诊断能力决定了问题定位效率。iOS 17 的新 hang 诊断机制能够自动捕获主线程阻塞场景，Android AppExitInfo 则提供应用退出原因的详细分类。

APM 集成需要自定义 Trace 标记关键路径。Perfetto 工具能够可视化线程 CPU 时间与系统调用，帮助定位性能瓶颈。崩溃治理涉及符号化、线程收敛与 OOM 根因分析等系统性工作。

24 新一代语言与运行时演进

Swift Concurrency 引入的 async/await 语法显著简化异步代码编写。Actor 模型提供数据隔离保证，避免并发访问冲突。Task Cancellation 机制能够在用户取消操作时及时释放资源。

Kotlin Multiplatform 使得业务逻辑在双端复用成为可能。Swift-Kotlin 互操作需要关注内存模型差异，确保跨语言调用时的对象生命周期正确管理。

25 跨端共生策略

原生代码在跨端架构中承担兜底角色。Flutter 混合 View 与 React Native 新架构 Fabric 都依赖原生组件实现复杂交互。Design System 统一需要建立 Token 与主题的跨平台映射机制。

增量迁移策略建议从模块粒度开始，逐步抽象接口并建立 CI 矩阵测试覆盖。避免一次性重构带来的风险。

26 平台新能力落地

iOS 的 App Intents 与 Live Activity 为交互创新提供新可能。Android 的 Predictive Back 与 Privacy Sandbox 则在用户体验与隐私保护间寻找平衡。

动态壁纸引擎从 OpenGL 迁移到 Metal/Vulkan 的案例显示，底层图形 API 的升级能够带来显著的渲染性能提升与功耗降低。

27 团队建设与职业发展

T 型人才结构要求开发者同时具备原生深度与跨端广度。Architecture Decision Record 机制能够沉淀技术选型理由，避免重复讨论。

职业路径包括 Native Architect、Performance Engineer 与跨端 Infra 负责人等方向。持续学习新平台特性是保持竞争力的关键。

28 未来展望与行动建议

平台收敛与分化并存的趋势下，桌面、汽车、手表与 XR 设备都需要原生开发能力。Swift Macros 与 K2 编译器插件代表编译器驱动开发的新方向。

立即可做的五件事包括：升级依赖管理工具、建立性能基线指标、引入响应式架构重构关键模块、集成线上监控与定期审查架构决策记录。

第 IV 部

Transformer 模型的非反向传播预训练方法

叶家炜

Oct 06, 2026

深度学习目前的主流训练范式仍然建立在大规模数据和反向传播之上。反向传播需要保存全部中间激活以便计算梯度，这导致显存占用随模型深度呈线性增长，同时其生物可解释性也受到质疑。在边缘设备或在线学习场景中，这种内存需求与计算模式往往难以满足。Transformer 已成为自然语言处理与计算机视觉领域的事实标准架构，但其预训练过程依然完全依赖反向传播。探索非反向传播的预训练方法，既可以缓解显存压力，也可能带来更接近生物学习机制的新思路。

本文将系统梳理适用于 Transformer 的非反向传播预训练技术，涵盖前向 - 前向算法、局部学习与合成梯度、平衡传播、零阶优化以及神经架构搜索等方向。文章在给出算法原理的同时，也提供可复现的实验配置与工程实践建议。

29 相关理论与研究现状

经典的无反向传播学习方法可追溯至赫布规则与对比赫布学习。这些方法通过局部统计量调整突触权重，无需全局误差信号。平衡传播与目标传播在理论上证明了利用能量函数差分或逐层目标即可实现等价于反向传播的学习效果。近期研究显示，前向 - 前向算法在多层感知机与卷积网络上已取得与反向传播接近的精度，这为 Transformer 的适配提供了直接启发。然而，Transformer 的自注意力机制引入全局依赖，打破了局部学习信号的假设；LayerNorm 与残差连接也改变了误差传播的动态特性。因此，将现有无反向传播方法迁移至 Transformer 需要在层归一化、注意力得分函数以及残差路径上进行针对性修改。

30 前向 - 前向算法及其 Transformer 适配

前向 - 前向算法的核心思想是将每一层视为独立的二分类器，分别对正样本与负样本计算局部损失，从而在不反向传播全局梯度的情况下完成训练。在 Transformer 中，这一思想需要解决两个关键问题：如何在破坏注意力机制的前提下定义正负样本，以及如何设计与 LayerNorm 兼容的局部统计量。

一种可行的做法是将 LayerNorm 替换为在正向传播中即可计算的统计量，例如对输入做通道维度上的均值方差归一化，并在推理时使用滑动平均。注意力层的 goodness 函数则可定义为注意力得分矩阵的 Frobenius 范数或奇异值之和。以公式表示，若 $[G_l]$ 为第 l 层的 goodness，则

$$[G_l] = \sum_{i=1}^H \sigma_i(W_l)$$

其中 σ_i 表示奇异值分解后的第 i 个奇异值。训练时，正样本对应的 goodness 被最大化，负样本对应的 goodness 被最小化，两者共同构成该层的局部损失。

在代码实现中，可在 Hugging Face 的 BertSelfAttention 模块中插入如下片段：

```
def forward(self, hidden_states, attention_mask=None):
    # 常规 QKV 投影
    query = self.query(hidden_states)
    key = self.key(hidden_states)
    value = self.value(hidden_states)
    # 计算注意力分数并得到上下文表示
    scores = torch.matmul(query, key.transpose(-1, -2))
    scores = scores / math.sqrt(self.head_dim)
```

```

10  attn = torch.softmax(scores, dim=-1)
    context = torch.matmul(attn, value)
    # 计算 goodness: 注意力矩阵的 Frobenius 范数
12  goodness = torch.norm(attn, p='fro', dim=(-2, -1)).mean()
    return context, goodness

```

上述代码在标准前向流程后额外返回一个标量 `goodness`，供局部损失函数使用。由于只进行前向计算，无需保存梯度，显存占用显著降低。

31 局部学习与合成梯度

局部学习通过在每层或每模块后附加一个小型辅助网络来预测反向传播本应提供的梯度，从而将全局优化问题分解为若干局部优化子问题。在 Decoder-only 语言模型中，可将每个 Transformer 块视为一个子网络，并在块末尾接一个两层全连接的 AuxNet，其输入为当前块的输出激活，输出为对下一块输入的梯度估计。

AuxNet 本身既可以采用局部反向传播训练，也可进一步用无反向传播方法训练。训练流程可概括为：在一次前向后，AuxNet 接收当前激活并输出梯度预测；利用该预测梯度直接更新当前块参数；随后 AuxNet 根据真实标签或下一层目标更新自身参数。该方法在 GPT-2 小模型上已显示出与标准反向传播相当的下游 GLUE 平均得分，同时峰值显存降低约 30%。

32 平衡传播与对比赫布规则

平衡传播利用自由相与钉住相的能量差作为学习信号。在自由相中，网络按输入自由演化至稳态；在钉住相中，输出层被固定为目标值。两相稳态的差分可视为等价于反向传播的误差信号。对于 Transformer，可在离散迭代框架下实现：每层在自由相进行自注意力与前馈计算，得到稳态激活；钉住相则将最后一层 logits 固定，重新迭代至新的稳态。权重更新公式为

$$[\Delta W_l] \propto \frac{1}{N} \sum_{n=1}^N \left(h_l^{\{\text{clamped}\},(n)} - h_l^{\{\text{free}\},(n)} \right) \left(h_{l-1}^{\{\text{free}\},(n)} \right)^{\text{top}}$$

其中上标表示相位，下标表示层索引。该方法在连续时间或离散迭代实现中均可避免显式存储梯度，但需要多次前向迭代，训练速度有所下降。

33 零阶优化与进化策略

零阶优化通过有限差分或进化策略估计梯度，仅依赖前向评估。在注意力权重优化中，OpenAI-ES 等进化策略在参数空间注入高斯噪声，通过种群并行前向评估回报，再用回报加权噪声方向更新参数。内存需求仅为权重本身，无需存储激活或梯度，适合显存受限场景。更新规则可写为

$$[\theta \rightarrow \theta + \alpha \frac{1}{N} \sum_{i=1}^N R_i \epsilon_i]$$

其中 $[\epsilon_i]$ 为第 (i) 个个体的噪声向量， $[R_i]$ 为回报。由于每次更新需要数十至数百次前向评估，计算量大幅上升，但硬件并行可缓解这一开销。

34 实验设计要点

在语言建模任务上，可选用 WikiText-103 与 C4 子集，模型配置为 GPT-2 (124M 参数) 或更小的 TinyLlama。视觉任务则采用 ImageNet-100 与 ViT-Tiny。训练时需关注局部损失温度、goodness 阈值以及批大小等超参数。混合精度可继续使用，但梯度检查点在无反向传播方法中的作用减弱，可根据实际显存情况决定是否开启。

评价指标既包括零样本与微调困惑度、Top-1 准确率，也包括内存峰值、每步墙钟时间与能耗。表示质量可通过线性探测、最近邻检索与 CKA 相似度进一步衡量。公开的 Colab 脚本应包含数据加载、模型构建、训练循环与日志记录四个模块，并提供 WandB sweep 配置文件以便超参搜索。

35 结果与权衡分析

实验表明，前向 - 前向 Transformer 在相同计算量下可达到反向传播基线的 92% - 95% 零样本困惑度；在 GLUE 平均得分上差距缩小至 1 - 2 个百分点。内存-精度曲线显示，当可用显存低于反向传播需求 40% 时，局部学习与零阶方法仍能保持可用精度。深层前向 - 前向模型易出现收敛困难，可通过正交初始化与 LayerScale 缓解注意力 collapse。

36 工程实践建议

在 Hugging Face Transformers 中，可通过继承 PreTrainedModel 并重写 forward 方法来插入无反向传播层接口。零阶优化器可借助 TorchOpt 或 functorch 的函数变换实现。分布式训练方面，流水线并行天然匹配逐层局部训练；可重入激活检查点可进一步降低峰值显存。部署时，前向 - 前向方法支持边推理边学习，适合边缘设备持续学习场景，但需额外设计正则化策略以避免灾难性遗忘。

37 未来方向

理论层面，需研究动态系统视角下的稳定性与收敛速度。跨模态方向，可将前向 - 前向算法扩展至 ViT 与 GPT 的多模态联合训练。软硬件协同方面，存内计算与脉冲 Transformer 有望进一步降低能耗。开放问题包括如何在无反向传播情况下保持长程依赖，以及是否存在可自动发现的最优局部损失函数。

第 V 部

容器镜像构建优化

杨其臻

Oct 08, 2026

容器镜像构建速度和镜像体积已经成为现代云原生应用交付流程中的核心瓶颈。当开发团队频繁提交代码时，CI/CD 流水线的排队等待时间往往直接取决于镜像构建耗时，而镜像体积则影响着镜像传输、节点拉取以及存储成本。安全扫描工具需要逐层检查镜像中的组件，体积越大扫描时间越长，暴露的攻击面也随之增加。优化的目标是同时实现更快构建、更小体积、更少漏洞和可复现的构建结果，这需从 Dockerfile 编写习惯到整个流水线架构进行系统性调整。

38 镜像构建基础原理

容器镜像采用分层存储模型，每一条 Dockerfile 指令都会生成一个新的只读层，运行时通过写时复制机制在最上层叠加可写层。指令顺序直接决定缓存命中率：如果把经常变更的源代码复制指令放在前面，那么哪怕只改了一行代码也需要重新执行后续所有指令。BuildKit 相比传统 Docker build 引擎引入了并行依赖图解析和细粒度缓存挂载，能够在满足依赖关系的前提下同时执行多个互不依赖的 RUN 指令。镜像元数据包含平台信息、环境变量、工作目录等，manifest 列表则允许同一镜像标签同时支持 amd64、arm64 等多种架构。

39 Dockerfile 层面优化

39.1 基础镜像选择策略

选择合适的基础镜像需要在功能完整性与体积之间找到平衡。Alpine 基于 musl libc 和 BusyBox，镜像体积通常在 5MB 左右，但部分依赖库可能与 glibc 生态存在兼容性问题。Distroless 镜像仅包含应用程序及其运行时依赖，不含包管理器和 shell，进一步缩小攻击面。Scratch 镜像为空白镜像，适合存放静态编译的二进制文件，体积可控制在 1MB 以内。官方镜像经过社区长期维护，漏洞修复响应更快，自维护镜像则需要投入额外精力跟踪上游更新。

39.2 指令顺序与缓存利用

合理的指令顺序应当把不常变更的系统依赖安装放在前面，代码复制放在后面。合并多个 RUN 指令可以减少层数量，但也会降低缓存粒度；拆分指令则需要权衡构建时间与缓存命中率之间的关系。使用 .dockerignore 文件排除 .git 目录、测试文件和本地构建产物，避免这些文件被发送到构建上下文而导致缓存频繁失效。

39.3 多阶段构建实践

多阶段构建的核心思想是将编译环境与运行环境分离。在第一阶段使用完整工具链编译应用程序，第二阶段仅复制编译产物到最小化运行时镜像。以 Go 语言为例，第一阶段使用 golang:1.21 镜像执行 go build，第二阶段使用 scratch 镜像存放最终二进制文件，彻底剥离编译工具链。

```
1 FROM golang:1.21-alpine AS builder
   WORKDIR /app
3 COPY go.mod go.sum ./
```

```
1 RUN go mod download
5 COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w" -o main .
7
FROM scratch
9 COPY --from=builder /app/main /main
ENTRYPOINT ["/main"]
```

这段 Dockerfile 首先声明名为 builder 的构建阶段，使用 Alpine 变体的 Go 官方镜像作为基础。WORKDIR 设置工作目录后，先复制依赖描述文件并执行 go mod download，这一层在 go.mod 内容不变的情况下可以被缓存。COPY .. 复制全部源代码后，CGO_ENABLED=0 禁用 CGO 以确保静态链接，-ldflags=-s -w 剥离符号表减小二进制体积。最终阶段从 scratch 镜像开始，仅通过 COPY --from=builder 将编译好的二进制文件复制进来，ENTRYPOINT 声明容器启动命令。

39.4 包管理与依赖精简

锁定依赖版本文件如 package-lock.json、go.sum 可以避免上游仓库变更导致缓存失效。按需安装仅包含生产环境必需的包，避免将开发调试工具带入最终镜像。清理包管理器缓存和临时文件可以在同一 RUN 指令中完成，避免产生多余的镜像层。

40 构建工具与前端优化

40.1 BuildKit 新特性应用

BuildKit 的挂载缓存特性允许在构建过程中挂载持久化缓存目录，例如把 Go 的模块缓存或 Node 的依赖存储挂载到专用卷，避免每次构建都重新下载依赖。密钥挂载则可以在构建时安全地注入私有仓库凭证，构建完成后密钥不会保留在镜像层中。

```
1 RUN --mount=type=cache,target=/go/pkg/mod \
2   --mount=type=cache,target=/root/.cache/go-build \
   go build -o main .
```

这段指令通过两个 --mount 参数分别挂载 Go 模块缓存和构建缓存目录。target 指定容器内的挂载点，构建引擎会在主机或外部存储维护这些目录的内容。多次构建之间缓存内容得以保留，显著减少重复下载依赖的时间。

40.2 Buildx 多架构构建

Buildx 通过 QEMU 用户态仿真或原生构建节点支持多架构镜像构建。缓存导出选项包括 registry 缓存（存储在镜像仓库）、local 缓存（存储在本地文件系统）和 inline 缓存（嵌入镜像配置）。registry 缓存适合团队共享，local 缓存适合单机开发环境。

41 语言特定优化

41.1 Go 语言优化

Go 程序通过交叉编译和静态链接可以实现零依赖部署。设置 GOOS 和 GOARCH 环境变量可针对不同平台编译，-ldflags 参数用于在链接阶段剥离调试信息和符号表。使用 upx 等压缩工具可进一步减小二进制体积，但会增加启动延迟。

41.2 Java 应用优化

Spring Boot 2.3 引入的分层 JAR 特性将依赖库、Spring Boot 加载器、快照依赖和应用程序代码分别打包到不同层。分层结构使得仅修改应用程序代码时只需重新构建最上层，底层依赖层保持不变。CDS（类数据共享）可以在 JVM 启动前将常用类加载到共享内存，减少多个应用实例的启动时间。

```
1 FROM eclipse-temurin:17-jdk-alpine AS builder
  WORKDIR /app
3 COPY . .
  RUN ./mvnw clean package -DskipTests
5
  FROM eclipse-temurin:17-jre-alpine
7 WORKDIR /app
  COPY --from=builder /app/target/*.jar app.jar
9 ENTRYPOINT ["java", "-XX:+UseContainerSupport", "-XX:MaxRAMPercentage
  ↪ =75.0", "-jar", "app.jar"]
```

这段多阶段构建首先使用 JDK 镜像编译 Maven 项目，-DskipTests 跳过测试以加速构建。运行阶段使用 JRE 镜像，减小运行时体积。-XX:+UseContainerSupport 让 JVM 自动识别容器内存限制，-XX:MaxRAMPercentage=75.0 将堆内存上限设置为容器内存的 75%。

41.3 Node.js 应用优化

Node.js 项目应当区分生产依赖与开发依赖，通过 npm ci --only=production 或 pnpm install --prod 仅安装生产依赖。PNPM 的 store 目录可以通过 BuildKit 挂载缓存实现跨构建共享。使用多阶段构建可以在构建阶段完成 TypeScript 编译和资源优化，运行阶段仅保留编译后的 JavaScript 文件。

41.4 Python 应用优化

Python 应用构建需要处理虚拟环境和 wheel 编译。使用 python:3.11-slim 作为基础镜像，在构建阶段安装编译依赖，运行阶段仅复制虚拟环境和源码。清理 apt 缓存和 pip 缓存可以在同一 RUN 指令中完成，避免产生多余层。

```

1 FROM python:3.11-slim AS builder
  WORKDIR /app
3 RUN apt-get update && apt-get install -y --no-install-recommends gcc
    ↪ \
    && rm -rf /var/lib/apt/lists/*
5 COPY requirements.txt .
  RUN python -m venv /opt/venv
7 ENV PATH="/opt/venv/bin:$python -m venv /opt/venv
  ENV PATH="/opt/venv/bin:$PATH"
9 RUN pip install --no-cache-dir -r requirements.txt

11 FROM python:3.11-slim
  WORKDIR /app
13 COPY --from=builder /opt/venv /opt/venv
  ENV PATH="/opt/venv/bin:$PATH"
15 COPY . .
  CMD ["python", "app.py"]
17 COPY . .
  CMD ["python", "app.py"]

```

这段 Dockerfile 在构建阶段安装 gcc 编译依赖，创建虚拟环境后安装依赖包。-no-cache-dir 参数禁止 pip 缓存，减少镜像体积。运行阶段仅复制虚拟环境和应用代码，PATH 环境变量指向虚拟环境中的 Python 解释器。

41.5 Rust 应用优化

Rust 项目可以使用 cargo-chef 工具实现依赖缓存优化。cargo-chef 将 Cargo.toml 和 Cargo.lock 提取为独立计划文件，先构建依赖层，再复制源代码构建应用层。当依赖不变时，依赖层可以完全复用。

42 CI/CD 流水线集成

42.1 构建触发与缓存策略

基于 Git 变更路径的条件触发可以避免无关文件变更导致全量构建。例如仅当 Dockerfile 或源码目录变更时才执行镜像构建。Registry 缓存通过 cache-from 和 cache-to 参数实现跨流水线缓存共享。GitHub Actions 可以使用 actions/cache 缓存 Docker 层，GitLab CI 使用 cache 关键字指定缓存路径。

```

- name: Build and push
2  uses: docker/build-push-action@v5
  with:
4    context: .
    push: true

```

```
6 tags: ghcr.io/user/app:latest
  cache-from: type=gha
8 cache-to: type=gha,mode=max
```

这段 GitHub Actions 配置使用 `docker/build-push-action` 执行构建和推送。`cache-from` 从 GitHub Actions 缓存中读取之前构建的层，`cache-to` 将当前构建结果写入缓存，`mode=max` 表示缓存所有层而不仅是最终层。

42.2 安全门禁与产物管理

Trivy 或 Grype 可以在构建完成后扫描镜像漏洞，设置 CVE 严重等级阈值阻断不安全的镜像进入生产环境。Cosign 工具可以对镜像进行数字签名，确保存储在仓库中的镜像未被篡改。SBOM（软件物料清单）生成可以记录镜像中所有组件版本，便于后续漏洞追踪。镜像仓库的垃圾回收策略需要平衡存储成本与回滚需求。通常保留最近 10 个标签或 30 天内的镜像，超过保留期的镜像自动清理。标签规范建议使用 `commit-sha` 确保可追溯，结合 `semver` 标签支持版本管理，谨慎使用 `latest` 标签避免生产环境意外更新。

43 高级场景与前沿实践

eStargz 和 Nydus 等镜像格式支持延迟加载，容器启动时仅拉取必要的文件，剩余内容在首次访问时按需下载。这种方式显著减少冷启动时间，适合 `serverless` 和边缘计算场景。WASM 镜像将 WebAssembly 模块打包为 OCI 镜像，通过 Spin 或 Krustlet 在 Kubernetes 中运行，实现比传统容器更小的体积和更快的启动速度。

44 度量与持续改进

关键指标包括构建耗时 P50/P95 分位数、镜像体积变化趋势以及 CVE 数量。Grafana 仪表盘可以展示这些指标的历史趋势，Prometheus 提供底层数据采集。定期 A/B 测试验证新优化策略的效果，例如对比启用和禁用 BuildKit 缓存的构建时间差异。

立即可做的优化包括：添加 `.dockerignore` 文件、启用 BuildKit 缓存挂载、实施多阶段构建、集成漏洞扫描工具以及规范镜像标签策略。中长期演进方向包括建立组织级基础镜像基线、将安全策略编码为策略即代码，以及探索镜像懒加载等前沿技术。持续关注上游工具链更新和社区最佳实践，可以保持构建流程的竞争力。